

Graph Neural Networks Applied to Money Laundering Detection in Intelligent Information Systems

Ítalo Della Garza Silva
Universidade Federal de Lavras
Lavras, Minas Gerais, Brazil
italo.silva7@estudante.ufla.br

Luiz Henrique Andrade Correia
Universidade Federal de Lavras
Lavras, Minas Gerais, Brazil
lcorreia@ufla.br

Erick Galani Maziero
Universidade Federal de Lavras
Lavras, Minas Gerais, Brazil
egmaziero@gmail.com

ABSTRACT

Context: Financial crimes exist in all world countries, and one of the most recurrent ones is Money Laundering (MoL). This crime can harm the country's economy, increase criminality, and compromise social investments. Moreover, it can increase the investment risk factor, raising exchange and interest rates and causing high inflation. In recent years, financial institutions and government agencies have searched for solutions to detect MoL in financial transactions. **Problem:** Several institutions have employed naive IS for detect MoL. Most systems are based on rules and label a large transactions number as suspicious, which makes the decision process inaccurate and inefficient. **Solution:** The recent literature presents Graph Neural Networks (GNN) as a promising solution to illegal transaction detection. We applied the Node and Edge Neural Network (NENN) architecture to classification, using the attributes of both bank accounts (vertices) and transactions (edges). **IS Theory:** In the Intelligent Information Systems context, Machine Learning is a way to improve the decision-making ability of programs in IS. **Method:** The GCN, Skip-GCN, and NENN architectures were evaluated for the MoL detection problem, comparing two ways of representing transactions as graphs (transactions as vertices or edges). Also, was considered the performance of XGBoost and Softmax classifiers in the solution. **Summary of Results:** Results showed better performance when transactions represented the nodes. In addition, NENN+XGBoost was superior for higher class imbalance values, with an F1-score of $74,51 \pm 4,21\%$ to "illicit" transactions. **Contributions and impacts in the IS area:** This paper improves the decision-making ability of Anti-Money Laundering systems, helping the organization and efficiency of public and private institutions, and contributing to the fight against corruption. This theme is aligned with the GrandDSI-BR2016-2026.

CCS CONCEPTS

• **Social and professional topics** → **Financial crime**; • **Computing methodologies** → **Neural networks**.

KEYWORDS

anti-money laundering, graph neural networks, deep learning.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SBSI '23, May 29–June 01, 2023, Maceió, Brazil

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0759-9/23/05...\$15.00
<https://doi.org/10.1145/3592813.3592912>

ACM Reference Format:

Ítalo Della Garza Silva, Luiz Henrique Andrade Correia, and Erick Galani Maziero. 2023. Graph Neural Networks Applied to Money Laundering Detection in Intelligent Information Systems. In *XIX Brazilian Symposium on Information Systems (SBSI '23)*, May 29–June 01, 2023, Maceió, Brazil. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3592813.3592912>

1 INTRODUÇÃO

Os crimes financeiros são um problema para todos os países, e cabe a determinadas agências federais e instituições financeiras identificar tais crimes, sendo que esse trabalho é feito através de uma série de processos investigativos bem-estabelecidos. Define-se "crime financeiro" por aquele praticado por uma entidade através do mercado financeiro, buscando ganho próprio em detrimento de outras entidades [7]. Dentre as principais atividades que se enquadram na categoria, vale destacar a Fraude Financeira, Sonegação de Impostos, Falsificação de Moedas, Manipulação de Mercado, e Lavagem de Dinheiro [12]. A Lavagem de Dinheiro pode ser definida como a ação de esconder dados que comprovem a ilegalidade do patrimônio obtido através de crimes, de forma a prevenir a aplicação dos devidos processos legais e a possibilidade de uso dos dados como prova do crime. A 11ª edição do Relatório Global de Fraude e Risco da Kroll mostra que a Lavagem de Dinheiro é um dos incidentes que mais afetaram as organizações em 2019 [17].

Dentre os Grandes Desafios de Pesquisa em Sistemas de Informação para o período 2016–2026, se encontra o uso de Sistemas de Informação na promoção da transparência, que é motivado também pela luta contra a corrupção [3], alinhado ao tema deste trabalho. Além disso, outro grande desafio é a integração e o aprimoramento de ecossistemas digitais – tanto de entidades privadas quanto governamentais – incluindo tecnologias emergentes e melhorando a eficiência e a organização no seu gerenciamento de processos de negócios [21], que também é objetivo deste trabalho.

Atualmente, tanto instituições financeiras quanto agências governamentais usam Inteligência Artificial em sistemas de detecção de Lavagem de Dinheiro. Considerando o contexto de Sistemas de Informação, pode-se afirmar que tais sistemas de detecção compõem os Sistemas de Informação Inteligentes (SSI) dessas organizações, uma vez que integram conjuntos de dados e Inteligência Artificial para auxiliar no cumprimento de uma tarefa de decisão. No entanto, a maior parte desses sistemas tende a ser simplista e baseado em regras [15], geralmente gerando um grande número de transações suspeitas e tornando a tarefa de detecção ineficiente.

A literatura tem apresentado várias alternativas aos sistemas baseados em regras, como sistemas baseados em Redes Neurais de Grafos (*Graph Neural Networks* – GNN). A principal motivação para aplicar GNNs nesse contexto é a possibilidade de se associar atributos de transações ou contas financeiras vizinhas, i.e., que estejam

ligadas direta ou indiretamente por meio de uma transação ou conta em comum, e com isso minerar informações sobre o contexto das transações a fim de tornar mais acurada a detecção das transações ilegais.

Existem duas maneiras de se representar dados de transações financeiras através de grafos. Uma opção é fazer com que os vértices do grafo sejam as transações financeiras. Essa forma é bastante utilizada quando o conjunto de dados vêm de um sistema de criptomoedas, como o Bitcoin Blockchain [25], e, nesse caso, as arestas do grafo representam o fluxo monetário. Para representar dados de transações bancárias comuns, é possível ligar as transações (vértices) entre si quando ambos tiverem a mesma origem ou destino [22]. A outra opção para representar o conjunto de dados é fazer com que os vértices sejam as contas bancárias e as arestas sejam as transações. Dessa maneira, é possível utilizar tanto os atributos das próprias contas financeiras quanto os atributos das transações e definir como a arquitetura de GNN irá incorporar os dados das arestas e vértices vizinhos separadamente. No entanto, nesse cenário, é necessário utilizar uma arquitetura de GNN que seja capaz de gerar *embeddings* para as arestas.

Um dos grandes desafios envolvendo GNNs aplicadas à detecção de Lavagem de Dinheiro é o desbalanceamento de classe [22], ou seja, o número de instâncias de uma classe é muito superior ao da outra classe (quando se trata de classificação binária). Isso ocorre porque, em ambientes de transação financeira, existem muito mais transações lícitas do que ilícitas. Isso afeta negativamente o treinamento do modelo e consequentemente diminui sua qualidade em uma aplicação no mundo real. A taxa de desbalanceamento de classe em um conjunto de dados para classificação binária pode ser definida como a razão entre o número total de instâncias e o número de instâncias da classe minoritária.

Este trabalho avalia a performance de modelos de GNN para o problema de detecção automática de Lavagem de Dinheiro em um conjunto de dados de transação financeira, classificando cada transação como "lícita" ou "ilícita", e comparando os resultados em ambas as maneiras de se representar o conjunto de dados em grafos (os vértices como transações financeiras e os vértices como contas financeiras), em diferentes níveis de desbalanceamento de classe.

Este artigo está organizado como descrito a seguir. A Seção 2 apresenta a base teórica necessária para a compreensão dos experimentos executados. Os trabalhos relacionados ao tema do artigo são apresentados na Seção 3. A Seção 4 descreve a metodologia adotada neste estudo. As Seções 5 e 6 listam e discutem, respectivamente, os resultados obtidos e seu impacto no contexto de Sistemas de Informação. Por fim, a Seção 7 conclui este trabalho e lista possíveis trabalhos futuros.

2 REFERENCIAL TEÓRICO

Esta seção apresenta o conceito de "Anti-Lavagem de Dinheiro" e a evolução da Inteligência Artificial aplicada a esta área. Também explana sobre as Redes Neurais de Grafos e cada variação desse subtipo de rede neural utilizada neste trabalho.

2.1 Anti-Lavagem de Dinheiro (*Anti-Money Laundering*)

Os sistemas Anti-Lavagem de Dinheiro (*Anti-Money Laundering* – AML) são implementados por vários tipos de instituições financeiras

e agências governamentais para prevenir atividades de Lavagem de Dinheiro em seu sistema financeiro [15]. Um sistema AML de uma instituição precisa ser suficientemente eficaz para prevenir que a existência de transações aprovadas em seu sistema sejam rotuladas como "ilegais" futuramente pelo sistema de outra instituição. Quando isso ocorre, o sistema de segurança da instituição pode estar comprometido, consequentemente afetando sua reputação e valor de mercado. Para lidar com isso, a maioria dos sistemas de segurança no mundo seguem as recomendações da *Financial Action Task Force* (FATF), chamadas de "Quarenta Recomendações da FATF" (*FATF 40 Recommendations*) [10], cujo desenvolvimento foi iniciado em 1990. Os sistemas de segurança que seguem as regras da FATF geram uma série de relatórios de atividade suspeita (*Suspicious Activity Reports* – SAR), os quais contêm informações sobre as transações, contas envolvidas e o tipo de atividade suspeita.

Esses sistemas baseados em regras normalmente geram um grande número de atividades suspeitas, sendo que a análise humana posterior é frequentemente necessária para filtrar as transações com um potencial real de serem suspeitas. Logo, a detecção leva um tempo considerável para ser executada, sendo necessário um gasto financeiro com os especialistas que realizam a análise posterior. Para solucionar esses problemas, a literatura apresenta várias soluções propondo técnicas alternativas de Inteligência Artificial para realizar a detecção de AML. Uma das abordagens é a Análise de Redes [8, 11], que pode ser aplicada a dados financeiros transacionais de forma a se obter links ocultos ou diretos de um nó no qual já foi detectada a Lavagem de Dinheiro.

Outra alternativa é a Análise de Links [20], que trata os dados financeiros como um grafo conexo e avalia as relações entre os nós desse grafo. A Detecção de *Outliers* [18, 19] busca detectar transações com um comportamento (i.e, seu conjunto de atributos) consideravelmente diferente do restante das transações (transações "anormais"). Muitos trabalhos também fazem uso de métodos de *Machine Learning* para classificação/*scoring* de risco, usando o resultado como base para detectar as transações suspeitas [5]. Por fim, há também o uso de Aprendizado de Grafos aplicado à detecção de Lavagem de Dinheiro [1], que compreende majoritariamente Redes Neurais de Grafos e é o foco deste trabalho.

A principal motivação da aplicação de Redes Neurais de Grafos na detecção de atividades suspeitas, é a possibilidade de se representar um conjunto de transações através de diferentes tipos de grafos [24], fazendo com que a detecção, ao avaliar uma transação, considere o contexto no qual ela está inserida, i.e, sua vizinhança e suas conexões na rede de transações.

2.2 Redes Neurais de Grafos

De acordo com [14], uma Rede Neural de Grafo (*Graph Neural Network* - GNN) é uma rede neural usada para lidar com dados estruturados em grafo. Nessa maneira de se representar os dados, cada vértice ou aresta (ou ambos) contém um vetor de atributos. Uma GNN recebe e processa esse grafo de dados, i.e, além do conjunto de atributos como entrada, que é comum a todas as redes neurais, ela receberá informações sobre como cada dado se liga a outros dados do conjunto. Essa informação pode ser representada por uma matriz de adjacência. A cada passo, a rede combina a informação de cada instância com a informação de seu respectivo vizinho no grafo, gerando um vetor *embedding*. Essa operação é chamada de

"message passing". É possível representar um passo do "message passing" de uma GNN pela Equação 1.

$$\begin{aligned} h_u^{(k+1)} &= \text{ATUALIZA}^{(k)} \left(h_u^{(k)}, \text{AGREGA}^{(k)} \left(\left\{ h_v^{(k)}, \forall v \in \mathcal{N}(u) \right\} \right) \right) \\ &= \text{ATUALIZA}^{(k)} \left(h_u^{(k)}, m_{\mathcal{N}(u)}^{(k)} \right). \end{aligned} \quad (1)$$

Dessa forma, os *embeddings* de todos os vértices v , na iteração k ($h_v^{(k)}$), sendo que v é vizinho do vértice u , são agregados por uma função AGREGA qualquer. O resultado é combinado com o *embedding* de u na mesma iteração ($h_u^{(k)}$), e então atualizados por uma função ATUALIZA qualquer, gerando o novo vetor *embedding* de u para a iteração $k + 1$. No primeiro passo, os vetores *embeddings* são os próprios vetores do conjunto de dados original. O grafo computacional da Figura 1 ilustra um exemplo do processo de agregação.

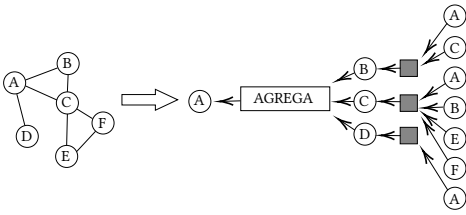


Figure 1: Grafo computacional para dois passos do message passing . Adaptado de [14].

O processo definido acima é a base de uma GNN. As arquiteturas específicas de GNN utilizadas neste trabalho são descritas a seguir.

2.2.1 Rede Convolutacional de Grafo. Proposta por [16], essa variação é uma das implementações mais comuns e simples de GNN, que realiza operações de convolução baseadas nas conexões entre os diferentes vértices do grafo. A Rede Convolutacional de Grafo (Graph Convolutional Network - GCN) agrega os vetores *embedding* dos vértices u e v normalizando cada *embedding* através do produto entre os graus de u e v ($|\mathcal{N}(u)|$ e $|\mathcal{N}(v)|$), respectivamente. O resultado é multiplicado pelo conjunto de pesos $W^{(k)}$ inserido da função de ativação σ (pode ser usada uma *Rectified Linear Unit* – ReLU – ou uma função sigmóide, por exemplo). Esse processo pode ser representado pela Equação 2. Para realizar a classificação, os *embeddings* da última camada são inseridos em uma camada linear seguida de uma camada *Softmax*.

$$h_u^{(k+1)} = \sigma \left(W^{(k)} \sum_{v \in \mathcal{N}(u) \cup \{u\}} \frac{h_v}{\sqrt{|\mathcal{N}(u)| |\mathcal{N}(v)|}} \right) \quad (2)$$

2.2.2 Skip-GCN. Uma variação "skip" da GCN comum foi apresentada por [25], e consiste na adição de uma conexão "skip" para amenizar o problema da degradação progressiva causada pela profundidade da rede neural. Essa conexão alimenta as camadas profundas da rede neural com os valores de entrada. Esse processo é mostrado na Equação 3

$$h_u^{(k)} = h_u^{(k-1)} + \tilde{W}x_u, \quad (3)$$

onde x_u é o vetor de atributos do nó u e $\tilde{W}x_u$ é a matriz de pesos da conexão "skip".

2.2.3 Rede Neural de Nós e Arestas. Introduzida por [26], a Rede Neural de Grafos com Nós e Arestas (Node and Edge Neural Networks - NENN) utiliza em suas operações tanto atributos de nós quanto de arestas, gerando *hidden embeddings* para ambos. Para compreender a NENN, é necessário definir os seguintes conceitos sobre vizinhança em grafos:

- **Vizinhos Baseados em Nó:** Para um grafo $G = (V, E)$, os vizinhos baseados em nó $\mathcal{N}_i$ de um nó i são os nós ligados diretamente a este por uma aresta, e o próprio nó i . Para uma aresta j desse mesmo grafo, seus vizinhos baseados em nó $\mathcal{N}_j$ são os nós conectados por essa aresta.
- **Vizinhos Baseados em Aresta:** Para um grafo $G = (V, E)$, os vizinhos baseados em aresta e_i de um nó i são as arestas que se conectam a esse nó. Para uma aresta j desse mesmo grafo, os seus vizinhos baseados em aresta são as arestas que se conectam aos nós conectados por esta aresta, e a própria aresta j .

A NENN possui dois tipos de camadas. As Camadas de Atenção no Nível de Nó (Node-Level Attention Layer) geram *embeddings* para os nós através de seus vizinhos baseados em nó e em aresta. Já as Camadas de Atenção no Nível de Aresta (Edge-Level Attention Layer) geram os *embeddings* para as arestas.

A Camada de Atenção no Nível de Nó calcula um coeficiente de importância α_{ij}^n de cada nó j para cada nó i . Esse coeficiente, para um dado nó i é obtido multiplicando-se os conjuntos de atributos x_i do nó i e x_j do nó j pela matriz de pesos $W_n \in \mathbb{R}^{d_v^l + d_v^{l+1}}$, sendo d_v^l o tamanho do *embedding* de um nó na camada l . Os resultados dessas multiplicações sofrem uma concatenação (nesta seção representados pelo símbolo $\parallel$) e multiplicado pelo vetor de parâmetros $a_n^T \in \mathbb{R}^{2d_v^{l+1}}$ de uma Rede Alimentada Adiante de camada única. Em seguida, o resultado passa pela função de ativação σ (p. ex. LeakyReLU), e é normalizado via *Softmax*, conforme mostra a Equação 4.

$$\alpha_{ij}^n = \frac{\exp(\sigma(a_n^T [W_n x_i \parallel W_n x_j]))}{\sum_{k \in \mathcal{N}_i} \exp(\sigma(a_n^T [W_n x_i \parallel W_n x_k]))} \quad (4)$$

Outro coeficiente de importância calculado por esta camada é o α_{ik}^e , que quantifica a importância de cada aresta k para cada nó i . Multiplica-se o conjunto de atributos e_k da aresta k pela matriz de pesos $W_e \in \mathbb{R}^{d_e^l + d_e^{l+1}}$ (sendo d_e^l o tamanho do *embedding* da aresta e na camada l). As multiplicações são então concatenadas e multiplicadas pelo vetor de parâmetros $a_e^T \in \mathbb{R}^{d_v^{l+1} + d_e^{l+1}}$ de uma Rede Alimentada Adiante de camada única. Por fim, o resultado é ativado pela função σ e normalizado via *Softmax*, como ilustrado na Equação 5.

$$\alpha_{ik}^e = \frac{\exp(\sigma(a_e^T [W_e x_i \parallel W_e e_k]))}{\sum_{j \in \mathcal{N}_i} \exp(\sigma(a_e^T [W_e x_i \parallel W_e e_j]))} \quad (5)$$

É calculada então uma média dos valores de entrada (já multiplicados pelas matrizes de peso da camada) ponderadas pelas importâncias obtidas, gerando o conjunto de valores $x_{\mathcal{N}_i}$ e x_{e_i} , conforme mostram as Equações 6 e 7.

$$x_{N_i} = \sigma \left(W_n \cdot \text{MEDIA} \left(\left\{ \alpha_{ij}^n x_j, \forall j \in N_i \right\} \right) \right) \quad (6)$$

$$x_{e_i} = \sigma \left(W_e \cdot \text{MEDIA} \left(\left\{ \alpha_{ik}^e e_k, \forall k \in e_i \right\} \right) \right) \quad (7)$$

Esses conjuntos de valores são então concatenados para formar o *embedding* de saída da camada, conforme mostra a Equação 8.

$$x_i^{(l+1)} = x_{N_i}^{(l)} \parallel x_{e_i}^{(l)} \quad (8)$$

A Camada de Atenção no Nível de Aresta trabalha de maneira similar à Camada de Atenção no Nível de Nó, porém calculando as atenções e *embeddings* dos vizinhos de cada aresta. Dada uma aresta i (com um conjunto de atributos e_i) qualquer do grafo e uma aresta k vizinha de i (com um conjunto de atributos e_k), multiplicam-se seus conjuntos de atributos pela matriz de pesos W_e e concatenam-se os resultados. Esse valor é então multiplicado pelo vetor de parâmetros $q_e^T \in \mathbb{R}^{2d_e^{l+1}}$, ativado pela função σ e normalizado pela função *Softmax* para se obter a importância β_{ik}^e , como mostra a Equação 9

$$\beta_{ik}^e = \frac{\exp \left(\sigma \left(q_e^T [W_e e_i \parallel W_e e_k] \right) \right)}{\sum_{j \in e_i} \exp \left(\sigma \left(q_e^T [W_e e_i \parallel W_e e_j] \right) \right)} \quad (9)$$

Da mesma forma, calcula-se a importância β_{ij}^n de cada nó j (com um conjunto de atributos x_j) para cada aresta i (com um conjunto de atributos e_i). O conjunto de atributos x_j é multiplicado pela matriz de pesos W_n e o conjunto de atributos e_i é multiplicado pela matriz de pesos W_e . Os dois resultados são concatenados e multiplicados pelo vetor de parâmetros $q_n^T \in \mathbb{R}^{d_v^{l+1} + d_e^{l+1}}$. Ativa-se o resultado com a função σ e normaliza-se via *Softmax*, como ilustra a Equação 10.

$$\beta_{ij}^n = \frac{\exp \left(\sigma \left(q_n^T [W_e e_i \parallel W_n x_j] \right) \right)}{\sum_{k \in N_i} \exp \left(\sigma \left(q_n^T [W_e e_i \parallel W_n x_k] \right) \right)} \quad (10)$$

Ocorre então o cálculo das médias dos valores de entrada, já multiplicados pelas matrizes de peso, ponderadas pelas importâncias obtidas. Em seguida, ocorre a concatenação desses valores para gerar o *embedding*, como mostram as Equações 11 e 12.

$$e_{e_i} = \sigma \left(W_e \cdot \text{MEDIA} \left(\left\{ \beta_{ik}^e e_k, \forall k \in e_i \right\} \right) \right) \quad (11)$$

$$e_{N_i} = \sigma \left(W_n \cdot \text{MEDIA} \left(\left\{ \beta_{ij}^n x_j, \forall j \in N_i \right\} \right) \right) \quad (12)$$

$$e_i^{(l+1)} = e_{N_i}^{(l)} \parallel e_{e_i}^{(l)} \quad (13)$$

Os *embeddings* calculados em uma camada são usados como atributos pela camada seguinte, sejam eles associados aos nós ou às arestas. A Figura 2 ilustra um modelo simples baseado na arquitetura NENN.

3 TRABALHOS RELACIONADOS

Uma base de dados de grafo foi desenvolvida por [25] para detectar crimes financeiros baseados em dados do mundo real pela *Bitcoin Blockchain*. Para realizar a classificação nesses dados, foi utilizada uma GCN simples de duas camadas. Os vetores *embedding* gerados pela rede foram também posteriormente incluídos no conjunto de atributos, visando testar a influência da inclusão de contexto na informação em outros métodos de classificação, nomeadamente Regressão Logística, *Random Forest* e *Multi-Layer Perceptron*. A influência da temporalidade nos dados foi também testada usando uma abordagem recorrente de GCN. O melhor resultado foi obtido

pela *Random Forest* via vetores *embedding*. Nenhum dos métodos propostos foi capaz de lidar com o fechamento do Mercado Negro que ocorreu em um dos *timesteps* do banco de dados.

Diversas técnicas de Mineração de Dados foram testadas no contexto de Lavagem de Dinheiro por [6]. Algoritmos de busca em grafos (como Dijkstra e *Breadth-First Search*), mineração de padrões frequentes (como *Eclat* e *FP-Close*) e correspondência em grafos (VF2) foram aplicadas em três conjuntos de dados reais de transações bancárias, cada conjunto relacionado a uma investigação diferente realizada por uma agência governamental. O estudo apresentou resultados satisfatórios usando-se técnicas baseadas em grafo para o conjunto de dados utilizado.

Um conjunto de dados de contas ilegais foi gerado por [9] para transações financeiras da *Ethereum Blockchain* e uma lista de contas bancárias que realmente praticaram atividades ilícitas naquele cenário. Foi utilizado XGBoost para detectar tais contas ilegais, obtendo altas acurácia e área sob curva ROC (AUC). O método proposto, no entanto, é incapaz de detectar a lógica computacional dos *smart contracts*, realizados para ocultar as atividades ilícitas. Além disso, [9] abordaram a detecção de contas financeiras, enquanto esse trabalho aborda a detecção das transações em si.

Uma nova abordagem de GCN foi usada por [2] para lidar com a base de dados desenvolvida por [25]. A GCN padrão foi modificada para lidar com grafos direcionados, através do Laplaciano normalizado simétrico. Foi obtida alta acurácia, porém baixa revocação para o conjunto de transações "ilícitas". A abordagem também não leva em consideração a natureza temporal dos dados no conjunto de dados. Os autores também sugeriram que a inclusão da data e hora real de cada transação pode ser muito mais significativa para o aprendizado que os *timesteps* colocados no conjunto de dados.

Uma nova abordagem que utiliza *multi-task learning* para a geração de *embeddings* foi proposta por [4]. O sistema, conhecido como DELATOR, utiliza o modelo GraphSAGE para a obtenção dos *embeddings* através de ambos aprendizado não supervisionado e regressão, que são combinados para se gerar os *embeddings* de saída. Para a classificação, foram testados tanto um classificador único quanto dois classificadores combinados (o segundo classificador "filtra" os casos considerados suspeitos pelo primeiro, buscando uma maior precisão). O delator foi testado em uma base de dados real, com transações do banco Inter, sendo que os resultados obtidos foram maiores do que os *baselines* comparados.

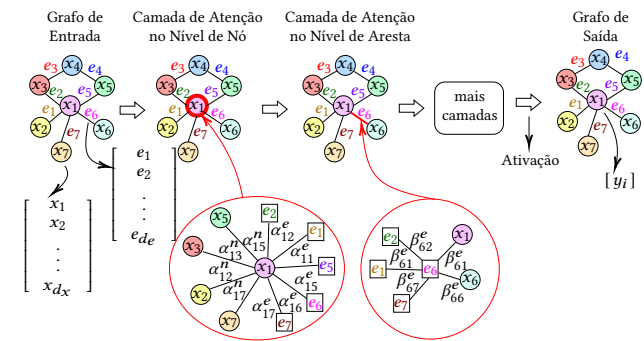


Figure 2: Arquitetura NENN. Adaptado de [26].

A maioria dos trabalhos citados utilizam bases de dados de transações feitas em criptomoedas, enquanto o foco deste trabalho são as transações bancárias. Os trabalhos que utilizam transações bancárias não divulgam o seu conjunto de dados, uma vez que a natureza de tais dados é sigilosa. Este trabalho utiliza dados gerados pelo simulador AMLSim, utilizado por [22]. No entanto, diferentemente dos trabalhos citados, este trabalho avalia a arquitetura NENN, que incorpora atributos de nós e arestas, no contexto da detecção de Lavagem de Dinheiro.

4 EXPERIMENTOS REALIZADOS

Para os experimentos realizados neste trabalho, foram utilizadas as arquiteturas GCN, Skip-GCN e NENN com uma camada totalmente conectada e um *Softmax* para fazer a classificação. Além disso, o algoritmo XGBoost foi treinado e testado com os *embeddings* gerados por essas três arquiteturas. Tanto a GCN quanto a Skip-GCN utilizadas possuem duas camadas e 16 neurônios nas camadas escondidas. a NENN foi construída com duas camadas de atenção de arestas (no início e no fim do conjunto de camadas escondidas) e uma camada de atenção de nós (no centro do conjunto de camadas escondidas).

Para as três arquiteturas o otimizador utilizado foi o *Adam*, sendo que a taxa de aprendizagem para as arquiteturas baseadas em GCN foi fixada em $1,0 \times 10^{-3}$, enquanto para a arquitetura NENN foi estabelecida em $1,0 \times 10^{-4}$. Foi utilizada a Entropia Cruzada com Pesos como função de erro. Para as arquiteturas GCN e Skip-GCN, os pesos foram fixados em 0,7 para a classe "ilícito" e 0,3 para a classe "lícito". A arquitetura NENN se mostrou consideravelmente sensível à variação da taxa de desbalanceamento e, por isso, foram fixados pesos diferentes para cada conjunto de dados testado. Para o AMLSim 1/3 não foram estabelecidos pesos. Para o AMLSim 1/5 foi fixado 0,6 para a classe "ilícito" e 0,4 para a classe "lícito". Para o AMLSim 1/10 foi fixado 0,85 para a classe "ilícito" e 0,15 para a classe "lícito". Por fim, para o AMLSim 1/20 foi fixado 0,96 para a classe "ilícito" e 0,04 para a classe "lícito". Foram testados vários pesos diferentes para cada conjunto de dados até que fosse encontrado um ponto de equilíbrio entre a precisão e a revocação do modelo. Cada modelo foi testado em cada conjunto de dados 100 vezes e os intervalos das métricas foram coletadas com 95% de confiança.

Os testes foram implementados em Python 3.8, sendo que as bibliotecas utilizadas para implementação dos modelos foram Pytorch 1.10 e Pytorch-Geometric 2.0.2, o XGBoost foi implementado com a biblioteca XGBoost 1.6.1, e as métricas foram coletadas via Scipy 1.8 e Scikit-learn 1.0.2. Os códigos utilizados encontram-se disponíveis *on-line* para fins de estudo¹

4.1 Conjuntos de Dados

Os dados utilizados neste trabalho foram gerados por um simulador de transações financeiras baseado em agentes, denominado AMLSim [23]. O AMLSim foi baseado no PaySim, porém contendo instruções específicas para a geração de transações com um comportamento típico de um esquema de Lavagem de Dinheiro. Através desse simulador, é possível configurar a quantidade total de *timesteps* (dias) da simulação, a data de início, o valor mínimo e máximo das transações, entre outros parâmetros estatísticos. Para este estudo, foram geradas transações diárias para o ano de 2020 (365

timesteps). Durante a simulação, são geradas informações sobre o identificador da conta, se é uma conta suspeita a priori, suas datas de abertura e fechamento, o depósito inicial, o identificador do padrão de comportamento, o identificador do banco, entre outros. As informações referentes à transação são o seu identificador, as contas de origem e destino, o tipo de transação, o dia, e se a transação é ou não suspeita. Para este trabalho, as informações utilizadas foram somente o valor transferido (*base_amt*), *timestep* (*tran_timestamp*) e tipo de transação (*tx_type*), para cada transação (além da informação sobre sua ilicitude (*is_sar*) e a conta de origem e destino, *orig_acct* e *bene_acct*, usadas na construção dos grafos), e informações de depósito inicial (*initial_deposit*) e categoria de comportamento (*tx_behavior_id*), para cada conta bancária. Uma amostra dos dados de transações e contas financeiras gerados pelo AMLSim, somente com os atributos utilizados neste trabalho, pode ser vista nas tabelas 1 e 2.

Table 1: Amostra de dados de transações bancárias geradas pelo AMLSim.

tran_id	orig_acct	bene_acct	tx_type	base_amt	tran_timestamp	is_sar
1	743	409	TRANSFER	727.06	2020-01-01	False
2	883	751	TRANSFER	397.13	2020-01-01	False
3	993	678	CASH_IN	933.56	2020-01-01	False
18	774	217	TRANSFER	199.22	2020-01-01	True

Table 2: Amostra de dados de contas bancárias geradas pelo AMLSim.

acct_id	initial_deposit	tx_behavior_id
7	66450.68	1
8	71285.46	1
9	93684.83	1
10	52558.76	1

De maneira similar ao que foi realizado por [22], os dados gerados foram posteriormente selecionados de forma a variar em cada conjunto a taxa de desbalanceamento de classe, para que fosse possível testar a performance de cada modelo em diferentes taxas de desbalanceamento. A Tabela 3 mostra cada conjunto de dados, sua respectiva taxa de desbalanceamento e a quantidade de exemplos que cada classe possui.

Table 3: Informações sobre os conjuntos de dados utilizados.

Conjunto de dados	# de transações lícitas	# de transações ilícitas	Total de transações	Taxa de desbalanceamento
AMLSim 1/3	2142	1071	3213	3
AMLSim 1/5	4284	1071	5355	5
AMLSim 1/10	9639	1071	10710	10
AMLSim 1/20	20349	1071	21420	20

Os dados gerados foram então normalizados via *MinMax*, sendo que os dados categóricos foram convertidos em múltiplas tabelas de dados binários via *one-hot-encoding*, ou seja, cada um correspondendo a uma categoria, e, para determinado atributo pertencente a uma determinada categoria, atribui-se 1 para a coluna referente àquela categoria e 0 para as demais. Os dados foram então convertidos para grafos. Para a testagem nas arquiteturas GCN e Skip-GCN, os vértices do grafo são as transações e dois vértices (ou transações)

¹Disponível em: <https://github.com/italodellagarza/SBSITests> Acesso: 09/12/2022

estão ligadas por uma aresta se tiverem alguma conta em comum entre elas, seja como depositante ou como beneficiário do depósito.

Para a testagem na arquitetura NENN, cada vértice do grafo é uma conta financeira e as arestas são as transações entre as diferentes contas. Dessa forma, os dados relativos as contas financeiras são utilizados somente nos testes feitos com a NENN. Todas as transações em todos os conjuntos de dados estão classificadas como "lícita" (0) ou "ilícita" (1). A Figura 3 ilustra a diferença entre os tipos de representações geradas para cada conjunto de dados.

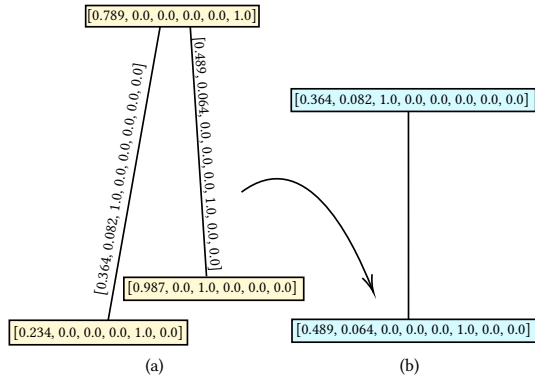


Figure 3: Diferença entre as representações via grafo do conjunto de dados utilizado. (a) conta bancária como um vértice do grafo e (b) transação como vértice do grafo.

4.2 Métricas

As métricas utilizadas neste trabalho foram Precisão (Equação 14), Revocação (Equação 15) e F1 (Equação 16). Todas as métricas foram calculadas considerando ambas as classes (macro), e também somente para a classe "ilícito". Para comparar os resultados, este trabalho foca na F1 da classe "ilícito", uma vez que ela consegue avaliar tanto a precisão quanto a revocação das classificações para esta classe. Ambas as métricas seriam importantes numa aplicação real, uma vez que, ao passo que se deseja obter o maior número possível de transações ilícitas, também é desejável que o número de falsos positivos seja o menor possível.

$$\text{Precisão} = \frac{TP}{TP + FP}, \quad (14)$$

$$\text{Revocação} = \frac{TP}{TP + FN}. \quad (15)$$

$$F1 = 2 * \frac{\text{Precisão} * \text{Revocação}}{\text{Precisão} + \text{Revocação}}. \quad (16)$$

5 RESULTADOS

A partir dos experimentos descritos na Seção 4, foi coletada a precisão, revocação e F1, tanto no nível macro como para a classe "ilícito", bem como seus respectivos intervalos de confiança. As métricas resultantes para cada modelo do experimento e separadas pelo conjunto de dados estão dispostas na Tabela 4.

Nota-se que, dentre todas métricas obtidas nos experimentos realizados nas bases de dados com taxa de desbalanceamento mais baixas, i.e., AMLSim 1/3 e AMLSim 1/5, os modelos baseados em GCN se mostraram superiores, com destaque para a combinação GCN+XGBoost, que obteve maior precisão e F1 para esses dois

conjuntos de dados. Também vale destacar as revocações obtidas pela arquitetura Skip-GCN, que foram mais altas que as demais, indicando que essa arquitetura é que obteve melhor cobertura para os conjuntos de dados supracitados, ainda que não tenha sido a mais precisa ao classificá-los. A arquitetura NENN obteve métricas mais baixas para esses conjuntos de dados (ainda que tenham sido satisfatórias), sobretudo nos testes em que a classificação foi realizada com o Softmax. Dessa forma, nota-se que a representação das transações como nós do grafo pode ter resultados positivos, ao menos para as taxas de desbalanceamento mais baixas.

A adição do XGBoost em todos os modelos provocou um aumento expressivo nos resultados. O XGBoost é um modelo que geralmente tem resultados positivos sobre dados tabulares [13] e sua inclusão nos modelos incrementou a capacidade dos mesmos ao lidar com a complexidade dos conjuntos de dados e sua resistência ao desequilíbrio de classe. Considerando-se os resultados sobre os conjuntos de dados AMLSim 1/10 e AMLSim 1/20, nota-se que a adição do XGBoost à arquitetura NENN fez com que sua F1, que de maneira geral era a pior em relação aos outros modelos, se tornasse a melhor.

5.1 Impacto do desbalanceamento de classe

O efeito do desbalanceamento começa a ser perceptível quando a taxa de desbalanceamento é igual a 10, sobretudo nos valores de precisão para a classe "ilícito". No cenário com as taxas de desbalanceamento mais altas, i.e., AMLSim 1/10 e AMLSim 1/20, a combinação NENN+XGBoost obteve precisão e F1 superiores às demais, exceto para o conjunto AMLSim 1/20, para o qual a maior precisão foi obtida pela combinação Skip-GCN+XGBoost. A arquitetura NENN combinada com a classificação por Softmax atingiu a melhor macro-revocação para o conjunto de dados AMLSim 1/20 e a melhor revocação para a classe "ilícito" no conjunto AMLSim 1/10. No entanto, a arquitetura apresenta baixa precisão em ambos os conjuntos de dados, o que indica que ela está atribuindo muitas instâncias para a classe "ilícito", sem necessariamente filtrar por aquelas que realmente são ilícitas. A baixa precisão dessa arquitetura afetou também a F1 do modelo, piorando-o em relação aos demais. Por fim, a GCN combinada com Softmax atingiu a melhor macro-revocação no conjunto AMLSim 1/10. É notável que a representação das transações como arestas do grafo se saiu melhor para taxas de desbalanceamento maiores.

A Figura 4 mostra o impacto do desbalanceamento de classe na F1 e na precisão para a classe "ilícito" para o modelo GCN, com e sem XGBoost. Nota-se uma queda significativa em ambas as métricas quando a taxa de desbalanceamento sobe de 5 para 10. Além disso, destaca-se o efeito positivo do uso do XGBoost como classificador para valores mais altos de desbalanceamento de classe, sobretudo na precisão.

Os efeitos do desbalanceamento de classe na F1 e na precisão para a classe "ilícito" para o modelo Skip-GCN são evidenciados pela Figura 5. Nota-se que as métricas foram muito similares às obtidas pelo modelo GCN, de forma que percebe-se o mesmo efeito positivo da adição do XGBoost ao modelo.

A Figura 6 mostra o efeito do desbalanceamento de classe na precisão e F1 para a classe "ilícito" do modelo NENN, com e sem a adição do XGBoost. Nesse modelo, quando o XGBoost não é usado, percebe-se uma queda mais acentuada nas métricas obtidas em

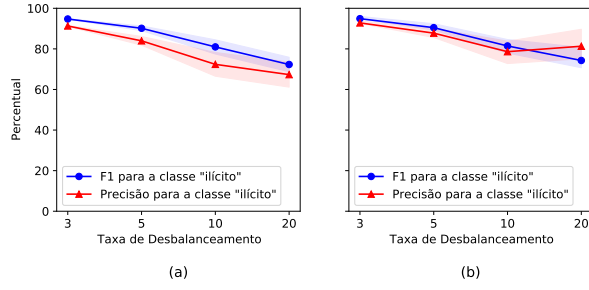


Figure 4: F1 e precisão para a classe "ilícito" do modelo GCN, com os classificadores Softmax (a) e XGBoost (b).

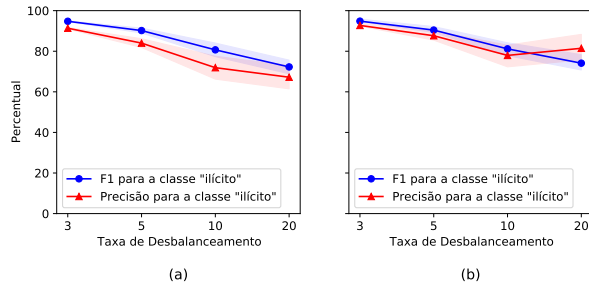


Figure 5: F1 e precisão para a classe "ilícito" do modelo Skip-GCN, com os classificadores Softmax (a) e XGBoost (b).

relação aos demais modelos. Logo, o modelo NENN, quando utilizado juntamente com o Softmax para classificação, se mostrou, no cenário abordado por este estudo, mais sensível aos efeitos do desbalanceamento de classe. Porém, com a adição do XGBoost, o modelo se mostra resistente a esses efeitos, inclusive superando os demais em termos de F1 e precisão. Pode-se dizer que a combinação NENN+XGBoost é a mais indicada para a tarefa de detecção apresentada neste estudo.

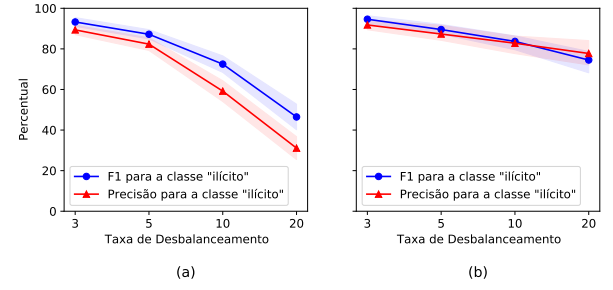


Figure 6: F1 e precisão para a classe "ilícito" do modelo NENN, com os classificadores Softmax (a) e XGBoost (b).

6 DISCUSSÃO

O combate à Lavagem de Dinheiro é um tema de extrema relevância para diversas organizações, tanto financeiras – para evitar prejuízos e perda de credibilidade perante a sociedade – quanto governamentais – para contribuir com a diminuição da corrupção e promover uma sociedade mais justa, transparente, e com menores índices de criminalidade. A substituição dos processos de detecção de Lavagem de Dinheiro, utilizados atualmente por essas instituições, é de suma importância para a constante evolução e inovação do seu ambiente organizacional, no que tange ao seu meio digital. Nesse sentido, pode-se afirmar que este trabalho contribui ao trazer uma potencial solução que, com as devidas adaptações, pode inovar os SSI utilizados para o combate à Lavagem de Dinheiro de diversas organizações.

Utilizando como exemplo os resultados do modelo NENN+XGBoost, têm-se aproximadamente 71% das transações ilícitas do conjunto de teste corretamente classificadas, sendo que aproximadamente 77% do total de instâncias do conjunto foram corretamente classificadas como ilícitas. Esses resultados equilibrados evidenciam uma solução que conseguiria, em um ambiente real, detectar uma imensa maioria de transações ilícitas e gerar apenas 30% de transações ilícitas, fazendo a discriminação manual de que são

Table 4: Métricas coletadas nos testes sobre os conjuntos de dados.

Arquitetura	AMLSim 1/3						AMLSim 1/5					
	Precisão		Revocação		F1		Precisão		Revocação		F1	
	macro	classe "ilícito"	macro	classe "ilícito"	macro	classe "ilícito"	macro	classe "ilícito"	macro	classe "ilícito"	macro	classe "ilícito"
GCN	95,31±0,12	91,30±0,23	97,27±0,06	98,43±0,00	96,21±0,09	94,73±0,13	91,67±1,05	83,97±2,14	96,60±0,28	97,32±0,43	93,87±0,73	90,15±1,15
Skip-GCN	95,34±0,17	91,35±0,34	97,28±0,08	98,44±0,00	96,23±0,14	94,76±0,18	91,70±1,07	84,01±2,16	96,62±0,30	97,35±0,28	93,90±0,76	90,19±1,19
NENN	94,15±1,49	89,36±2,34	96,34±1,70	97,52±3,00	95,13±1,54	93,26±2,14	90,34±1,61	82,34±3,10	94,14±1,60	92,70±3,18	92,07±1,42	87,20±2,28
GCN+XGBoost	95,79±0,65	92,80±0,99	96,98±1,00	97,10±2,03	96,35±0,77	94,89±1,10	93,13±1,26	87,76±2,39	95,23±1,48	93,35±2,94	94,12±1,18	90,46±1,93
Skip-GCN+XGBoost	95,74±0,63	92,72±1,14	96,94±0,89	97,04±1,86	96,30±0,69	94,83±0,98	93,08±1,24	87,64±2,49	95,23±1,38	93,40±2,89	94,10±1,06	90,42±1,73
NENN+XGBoost	95,39±1,36	91,79±2,44	97,00±1,24	97,65±2,08	96,14±1,26	94,61±1,74	92,75±2,16	87,32±4,33	94,47±1,41	91,91±2,79	93,57±1,57	89,53±2,52

Arquitetura	AMLSim 1/10						AMLSim 1/20					
	Precisão		Revocação		F1		Precisão		Revocação		F1	
	macro	classe "ilícito"	macro	classe "ilícito"	macro	classe "ilícito"	macro	classe "ilícito"	macro	classe "ilícito"	macro	classe "ilícito"
GCN	85,78±2,91	72,38±5,85	94,23±0,50	91,99±0,94	89,39±2,00	80,98±3,49	83,15±3,08	67,32±6,17	88,20±1,01	78,19±2,04	85,45±1,90	72,31±3,56
Skip-GCN	85,54±2,80	71,90±5,64	94,20±0,49	92,02±1,04	89,23±1,92	80,69±3,34	83,09±2,87	67,20±5,73	88,22±0,82	78,24±1,62	85,42±1,80	72,27±3,36
NENN	79,26±2,59	59,19±5,12	93,57±1,71	93,60±3,17	84,41±2,40	72,49±4,07	65,36±2,81	31,10±5,63	91,42±3,14	92,57±6,71	70,59±3,87	46,46±6,30
GCN+XGBoost	88,53±2,57	78,62±5,16	89,76±1,77	84,56±4,03	89,76±1,77	81,44±3,17	89,92±4,27	81,31±8,35	83,86±3,15	68,45±6,10	86,60±3,33	74,29±6,39
Skip-GCN+XGBoost	88,20±2,52	77,95±5,03	91,16±1,55	84,73±3,16	89,60±1,68	81,17±2,99	89,98±3,46	81,45±6,86	83,67±2,06	86,06±4,05	86,51±2,19	74,12±4,20
NENN+XGBoost	90,63±1,76	82,79±3,53	91,44±2,19	84,65±4,49	91,02±1,49	83,68±2,73	88,21±3,17	77,74±6,34	85,35±2,90	71,66±5,89	86,69±2,20	74,51±4,21

transações realmente ilícitas feita posteriormente diminua consideravelmente o consumo de tempo e recursos. Vale lembrar que este trabalho usou dados gerados por um simulador, e os resultados podem sofrer variações se os mesmos modelos forem aplicados a conjuntos de transações reais.

A principal limitação relacionada à solução proposta é o fato de a mesma ter sido treinada com dados fictícios, gerados por um simulador, os quais não trazem consigo todos os ruídos e particularidades de dados reais. Além disso, esta solução não aborda o desvio de conceito, outro fator presente em dados reais de transações bancárias. O desvio de conceito é a mudança nas características estatísticas dos dados ao longo do tempo.

7 CONCLUSÕES E TRABALHOS FUTUROS

Este trabalho comparou o desempenho de um conjunto de modelos de Redes Neurais de Grafos e também duas maneiras diferentes de representação das transações via grafos. De uma maneira geral, a representação das transações como nós do grafo pode ter resultados positivos. No entanto, para um maior desbalanceamento de classe, representar os nós como arestas do grafo pode ser bastante eficaz, sobretudo porque os *embeddings* são gerados tanto a partir dos dados da transação quanto a partir dos dados das contas financeiras.

Levando em consideração os resultados, conclui-se que, para taxas de desbalanceamento menores, os modelos baseados em GCN são mais apropriados. Para taxas de desbalanceamento maiores, a combinação NENN+XGBoost se mostra mais eficaz. Além disso, como a arquitetura NENN gera *embeddings* também para os nós, é possível facilmente adaptar o modelo para classificar as contas financeiras, sendo este um importante diferencial em uma situação real.

Como trabalhos futuros, serão incluídas novas arquiteturas neste experimento, inclusive arquiteturas propositalmente resistentes ao desequilíbrio de classe, além da inclusão de outros métodos de *Machine Learning* (como o XGBoost, por exemplo), sem o uso dos *embeddings*. Além disso, pretende-se acrescentar aos experimentos dados de transações financeiras reais, além dos dados gerados automaticamente, o que poderá afetar significativamente os resultados.

REFERENCES

- [1] Ismail Alarab and Simant Prakoonwit. 2022. Graph-Based LSTM for Anti-money Laundering: Experimenting Temporal Graph Convolutional Network with Bitcoin Data. *Neural Processing Letters* 54 (06 2022), 1–19. <https://doi.org/10.1007/s11063-022-10904-8>
- [2] Ismail Alarab, Simant Prakoonwit, and Mohamed Ikbil Nacer. 2020. Competence of Graph Convolutional Networks for Anti-Money Laundering in Bitcoin Blockchain. In *Proceedings of the 2020 5th International Conference on Machine Learning Technologies* (Beijing, China) (ICMLT 2020). Association for Computing Machinery, New York, NY, USA, 23–27. <https://doi.org/10.1145/3409073.3409080>
- [3] Renata Araujo. 2017. *Information Systems and the Open World Challenges*. Brazilian Computer Society (SBC), 42 – 51. <https://doi.org/10.5753/sbc.2884.0.4>
- [4] Henrique Assumpção, Fabrício Souza, Leandro Campos, Vinícius Pires, Paulo Almeida, and Fabrício Murai. 2022. Delator: Detecção Automática de Indícios de Lavagem de Dinheiro por Redes Neurais em Grafos de Transações. In *Anais do XI Brazilian Workshop on Social Network Analysis and Mining* (Niterói). SBC, Porto Alegre, RS, Brasil, 13–24. <https://doi.org/10.5753/brasnam.2022.223137>
- [5] Zhiyuan Chen, D. Van-Khoa Le, Ee Teoh, Amril Nazir, Ettikan Karuppiyah, and Kim Lam. 2018. Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: a review. *Knowledge and Information Systems* 57 (11 2018), 245–285. <https://doi.org/10.1007/s10115-017-1144-z>
- [6] Luis Fernando Carvalho Dias and Fernando Silva Parreiras. 2019. Comparing Data Mining Techniques for Anti-Money Laundering. In *Proceedings of the XV Brazilian Symposium on Information Systems* (Aracaju, Brazil) (SBSI'19). Association for Computing Machinery, New York, NY, USA, Article 73, 8 pages. <https://doi.org/10.1145/3330204.3330283>
- [7] Rafał Dreżewski, Jan Sepielak, and Wojciech Filipkowski. 2012. System supporting money laundering detection. *Digital Investigation* 9, 1 (2012), 8–21. <https://doi.org/10.1016/j.diin.2012.04.003>
- [8] Rafał Dreżewski, Jan Sepielak, and Wojciech Filipkowski. 2015. The application of social network analysis algorithms in a system supporting money laundering detection. *Information Sciences* 295 (2015), 18–32. <https://doi.org/10.1016/j.ins.2014.10.015>
- [9] Steven Farrugia, Joshua Ellul, and George Azzopardi. 2020. Detection of illicit accounts over the Ethereum blockchain. *Expert Systems with Applications* 150 (2020), 113318. <https://doi.org/10.1016/j.eswa.2020.113318>
- [10] FATF. 2003. FATF 40 Recommendations. <https://www.fatf-gafi.org/media/fatf/documents/FATF%20Standards%20-%202040%20Recommendations%20rc.pdf>
- [11] Andrea Fronzetti Colladon and Elisa Remondi. 2017. Using social network analysis to prevent money laundering. *Expert Systems with Applications* 67 (2017), 49–58. <https://doi.org/10.1016/j.eswa.2016.09.029>
- [12] Petter Gottschalk. 2010. Categories of financial crime. *Journal of Financial Crime* 17 (10 2010), 441–458. <https://doi.org/10.1108/13590791011082797>
- [13] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why do tree-based models still outperform deep learning on tabular data? <https://doi.org/10.48550/ARXIV.2207.08815>
- [14] William L. Hamilton. 2020. *Graph Representation Learning*. Number 46 in Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, San Rafael – CA, USA. <https://doi.org/10.2200/S01045ED1V01Y202009AIM046>
- [15] Jingguang Han, Yuyun Huang, Sha Liu, and Kieran Towey. 2020. Artificial intelligence for anti-money laundering: a review and extension. *Digital Finance* 2020 2:3 2, 3 (jun 2020), 211–239. <https://doi.org/10.1007/S42521-020-00023-1>
- [16] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Toulon, France, 14 pages.
- [17] Kroll. 2019. *Global Fraud and Risk Report 2019/20* (11 ed.). Technical Report. Kroll, Boston - MA, US. <https://www.kroll.com/en/insights/publications/global-fraud-and-risk-report-2019>
- [18] Asma S. Larik and Sajjad Haider. 2011. Clustering based anomalous transaction reporting. *Procedia Computer Science* 3 (2011), 606–610. <https://doi.org/10.1016/j.procs.2010.12.101> World Conference on Information Technology.
- [19] Nhien-An Le-Khac, Sammer Markos, and Tahar Kechadi. 2009. Towards a New Data Mining-Based Approach for Anti-Money Laundering in an International Investment Bank. In *Lecture Notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering*, Vol. 31. Springer, Albany, NY, USA, 77–84. https://doi.org/10.1007/978-3-642-11534-9_8
- [20] Edgar Alonso Lopez-Rojaz and Stefan Axelsson. 2012. Money Laundering Detection using Synthetic Data. In *Linköping Electronic Conference Proceedings*, No. 71. Linköping University Electronic Press, Linköping, Sweden, 33 – 40.
- [21] Vanessa Nunes, Claudia Cappelli, and Célia Ralha. 2017. *Transparency in Information Systems*. Brazilian Computer Society (SBC), 73 – 89. <https://doi.org/10.5753/sbc.2884.0.7>
- [22] Ronald Pereira and Fabrício Murai. 2021. Quão efetivas são Redes Neurais baseadas em Grafos na Detecção de Fraude para Dados em Rede?. In *Anais do X Brazilian Workshop on Social Network Analysis and Mining*. SBC, Porto Alegre, RS, Brasil, 205–210. <https://doi.org/10.5753/brasnam.2021.16141>
- [23] Toyotaro Suzumura and Hiroki Kanezashi. 2021. Anti-Money Laundering Datasets: InPlusLab Anti-Money Laundering DataDatasets. <http://github.com/IBM/AMLSim>
- [24] Mark Weber, Jie Chen, Toyotaro Suzumura, Aldo Pareja, Tengfei Ma, Hiroki Kanezashi, Tim Kaler, Charles E. Leiserson, and Tao B. Schardl. 2018. Scalable Graph Learning for Anti-Money Laundering: A First Look. *CoRR abs/1812.00076* (2018), 7 pages.
- [25] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. 2019. Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics. , 7 pages. https://drive.google.com/drive/folders/1r_iYFJru-jdDgpb-KZ1N0Zathy2LD2
- [26] Yulei Yang and Dongsheng Li. 2020. NENN: Incorporate Node and Edge Features in Graph Neural Networks. In *Proceedings of The 12th Asian Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 129)*, Sinno Jialin Pan and Masashi Sugiyama (Eds.). PMLR, Bangkok, Thailand, 593–608. <https://proceedings.mlr.press/v129/yang20a.html>